



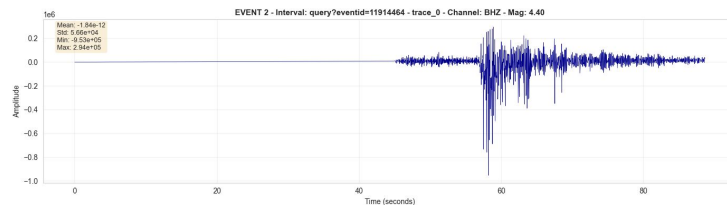
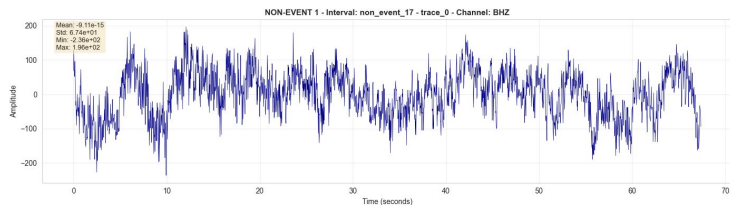
Detección de sismos mediante técnicas de TDA

Aaron Bernal Huanca

Materia: Topología aplicada y análisis topológico de datos 2do Cuatrimestre 2025
Profesor: Gabriel Minian

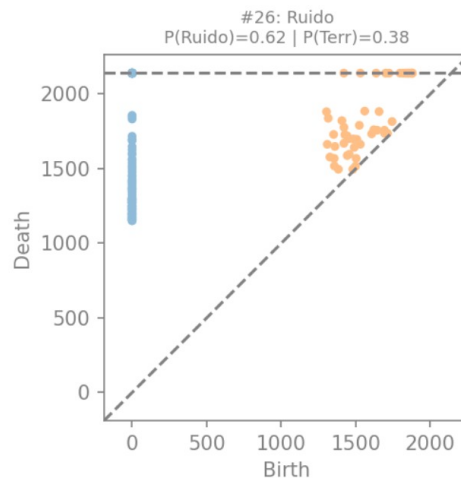
Objetivo

Dado una señal sismológica en un intervalo de tiempo, el objetivo poder decidir si se trata de un terremoto (clase 1) o ruido (clase 0), de forma robusta y eficiente. La idea general diseñar una herramienta de alerta temprana para apoyar a las actuales. Desde ya reducir daños.



¿Por qué TDA?

- "Classification of Single-lead Electrocardiograms: TDA Informed Machine Learning" by Matthew A. Reyna, et al. (2019)
- "Topological Data Analysis for Arrhythmia Detection Through Modular Neural Networks" by Meryll Dindin, Yuhei Umeda, Frederic Chazal (2020)





Fuente de datos

Estación GO01 de la red Chilean National Seismic Network disponible por The International Federation of Digital Seismograph Networks.

Link FDSN: <https://www.fdsn.org/networks/detail/C/>

API para la obtención de datos: ObsPy open-source Python framework for processing seismological data.

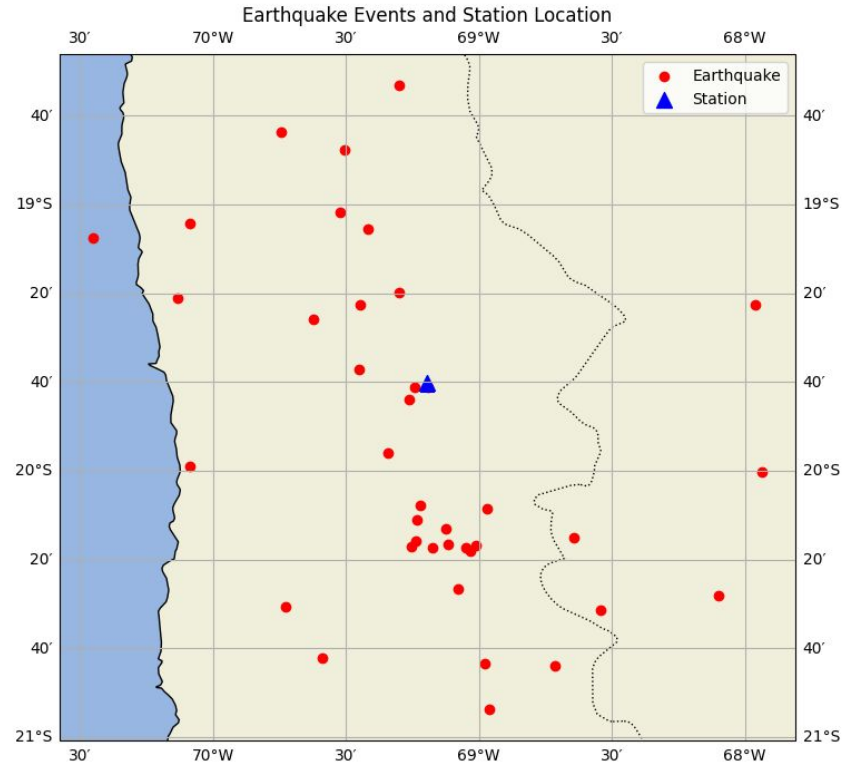
Fuente de datos

GO01 recopila de forma continua la actividad sísmológica con su sensor a 40Hz.

Para la búsqueda de hiperparámetros tomamos 400 señales entre 2023-11-01 y 2025-11-01.

50 son alrededor de un sismos. Y las 50 restantes son ruido.

La duración de cada intervalo oscila entre 65 y 90 segundos.



Arquitectura general

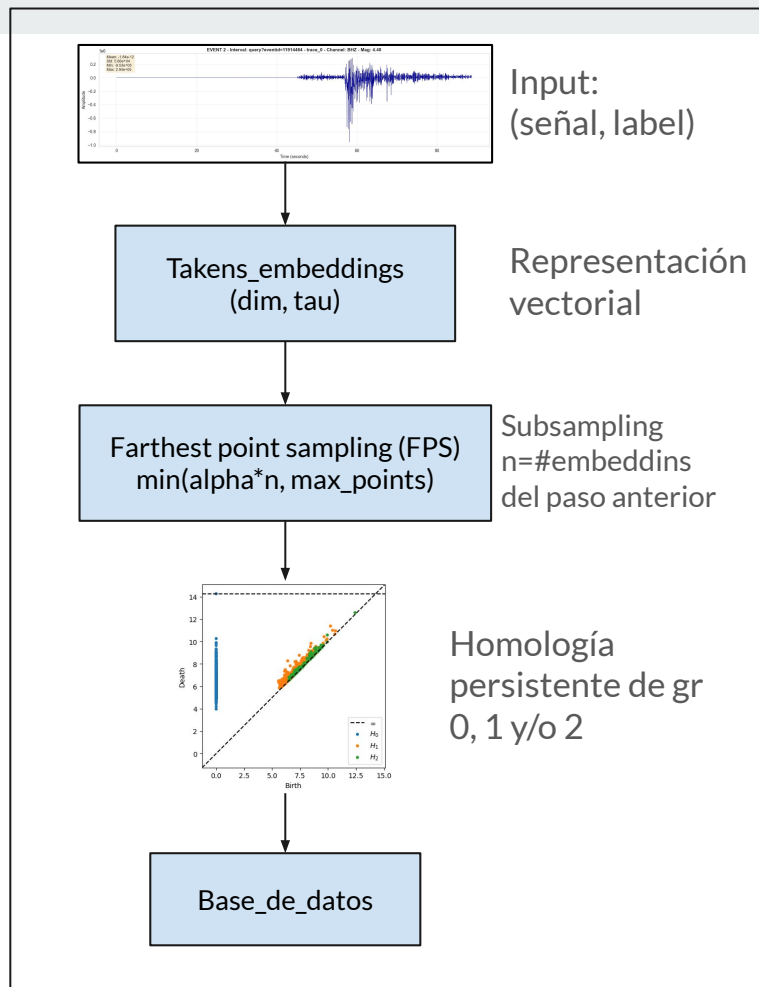
En el entrenamiento, se guarda una cantidad equilibrada de muestras por clase.

En la predicción,

- Input: señal sin label (clase)
- Output: probabilidad de ser clase 1

Para el output se calcula la distancia bottleneck o Wasserstein promedio por clase: (dist0, dist1)

$$\mathcal{P}(x \in clase1) = \frac{e^{dist0}}{e^{dist0} + e^{dist1}}$$



Arquitectura general

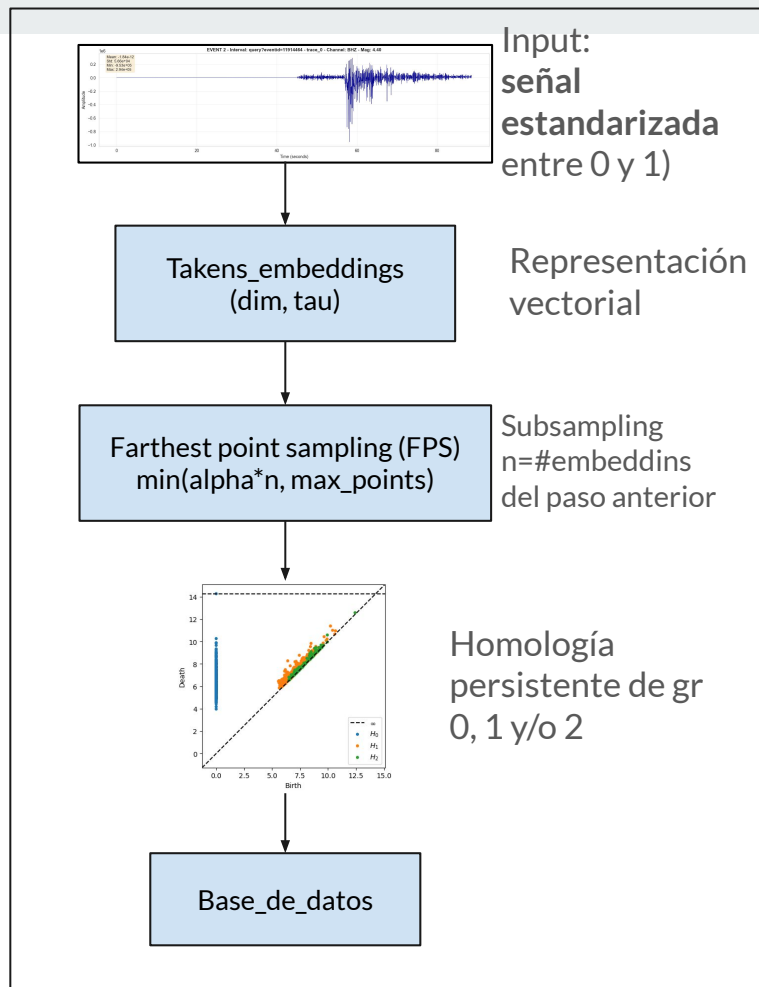
En el entrenamiento, se guarda una cantidad equilibrada de muestras por clase.

En la predicción,

- Input: señal sin label (clase)
- Output: probabilidad de ser clase 1

Para el output se calcula la distancia bottleneck o Wasserstein promedio por clase: (dist0, dist1)

$$\mathcal{P}(x \in clase1) = \frac{e^{dist0}}{e^{dist0} + e^{dist1}}$$



Búsqueda de hiperparámetros

Se realizó una búsqueda random de 100 combinaciones de la siguiente tabla, cada combinación se validó con k-fold cross validation (k=3). La elección de los valores se realizó de forma heurística para limitar la complejidad temporal. Por ejemplo,

- Nos limitamos H0 y H1.
- La distancia de bottleneck resultó muy costosa, para realizar la prueba nos limitamos a Wasserstein
- Sampleamos hasta 90 muestras por clase de la base de datos porque Wasserstein seguía siendo costoso.

Parámetro	Valores / Rango	Descripción
distance	wasserstein	Métrica de distancia entre diagramas de persistencia
weights	(1,), (1, 1), (2, 1), (1, 2)	Pesos para el cálculo de distancia
sample	[20, 30, 40]	Cantidad de diagramas a muestrear
thresh	[0,25, 0,37, ..., 1,40]	Umbral de homología persistente (10 valores)
alpha	[0,25, 0,33, ..., 0,99]	Proporción de submuestreo FPS (10 valores)
max_points	[100, 110, ..., 170]	Número máximo de puntos de Takens
seed	28	Semilla aleatoria para reproducibilidad

Parámetros adicionales para Takens Embedding:

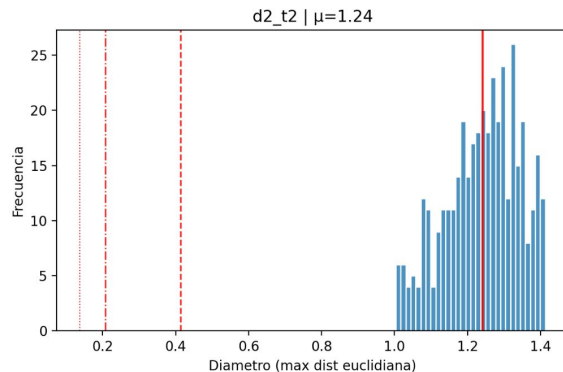
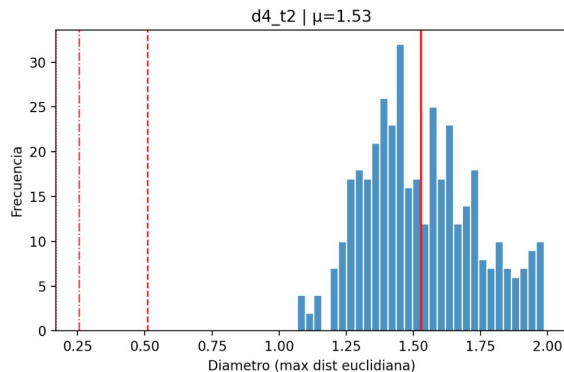
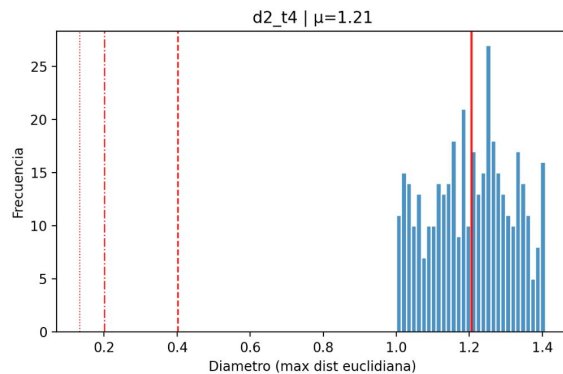
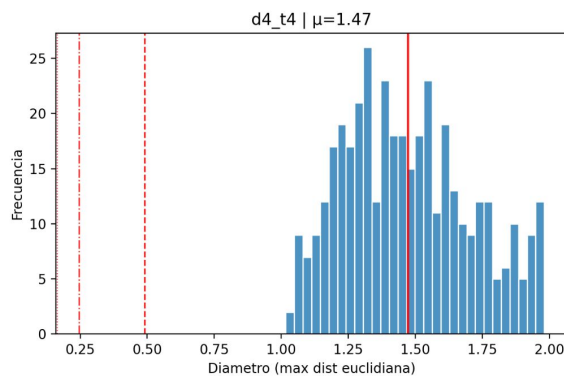
dim	[2, 4, 8, 16]	Dimensión del embedding de Takens
tau	[2, 4, 8, 16]	Retraso temporal (delay)
stride	[1, 2, 3, 4]	Paso para la ventana deslizante

Búsqueda de hiperparámetros - Thresh o épsilon

Distribución de **diámetros** de la nube de puntos en los datos de testeo.

Se prueba con distintos **dim** y **tau**.

Permite definir un rango de valores razonable.



Resultados

La configuración 1 obtuvo un AUCROC de 0.9228 lo cual es considerablemente bueno dado la robustez de esta métrica de performance.

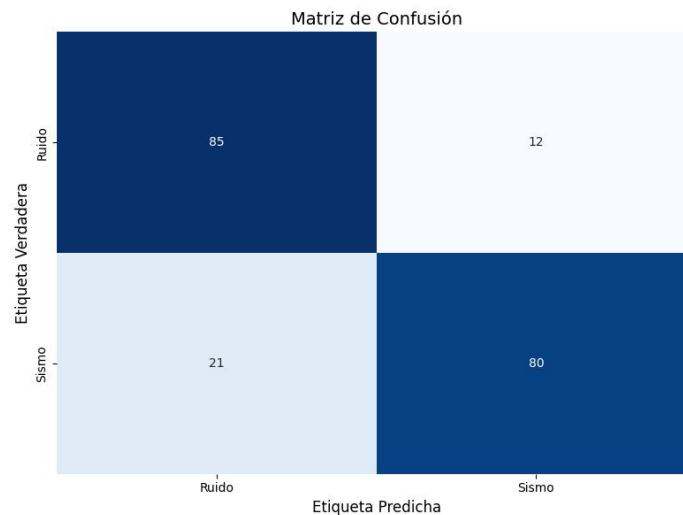
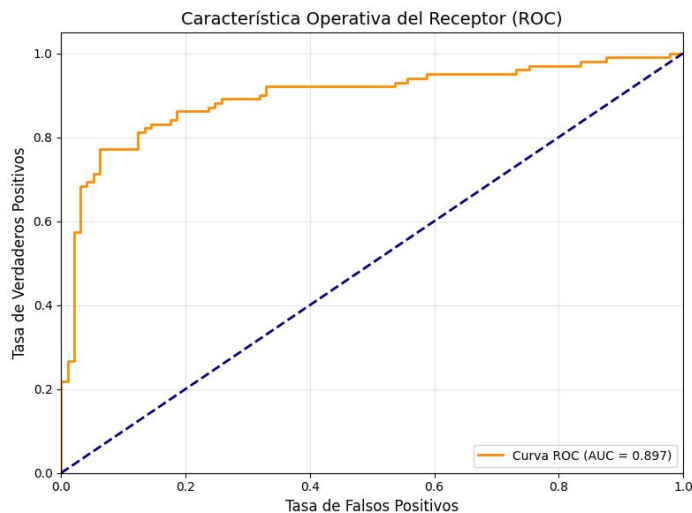
“Si seleccionamos al azar una instancia clase1 y otra clase0, el AUC ROC nos dice la probabilidad de que el modelo le asigne un puntaje más alto a la clase1.” Aprendizaje automático I - Pablo Brusco

Parámetro	Config 1	Config 2	Config 3	Config 4	Config 5
Fit Time (s)	1.155	0.741	1.579	1.071	1.077
Weights	(1, 1)	(1,)	(1,)	(1,)	(1, 1)
Threshold	1.272	1.272	0.894	1.272	1.272
Tau	4	4	16	8	8
Dim	2	2	4	2	2
Stride	4	4	4	4	4
Sample	40	30	60	40	40
Max_Points	140	170	160	150	120
Distance	wasserstein	wasserstein	wasserstein	wasserstein	wasserstein
Alpha	0.579	0.826	0.662	0.579	0.413
Mean AUC-ROC	0.8701	0.8691	0.8669	0.8641	0.8631
Std AUC-ROC	0.0083	0.0216	0.0101	0.0152	0.0097
Rank	1	2	3	4	5

Evaluación

La configuración 1 obtuvo un AUCROC de 0.897 lo cual es considerablemente bueno dado la robustez de esta métrica de performance.

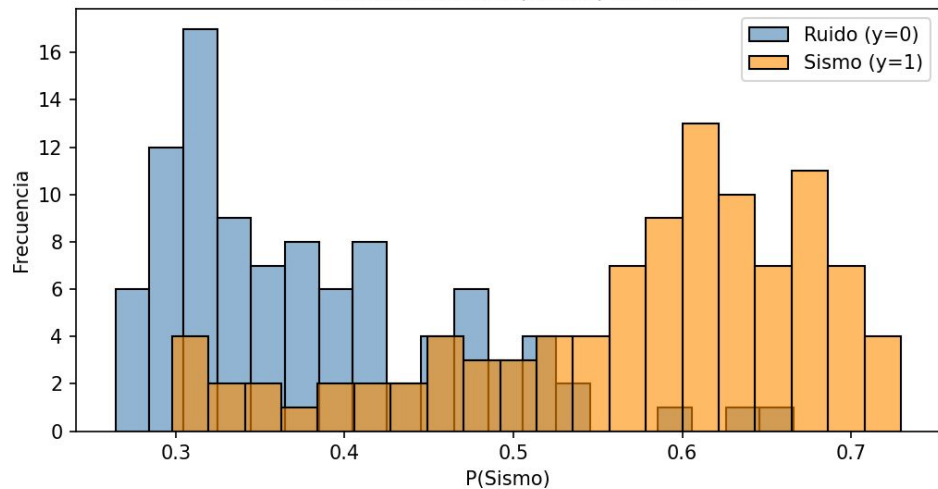
“Si seleccionamos al azar una instancia clase1 y otra clase0, el AUC ROC nos dice la probabilidad de que el modelo le asigne un puntaje más alto a la clase1.” Aprendizaje automático I - Pablo Brusco



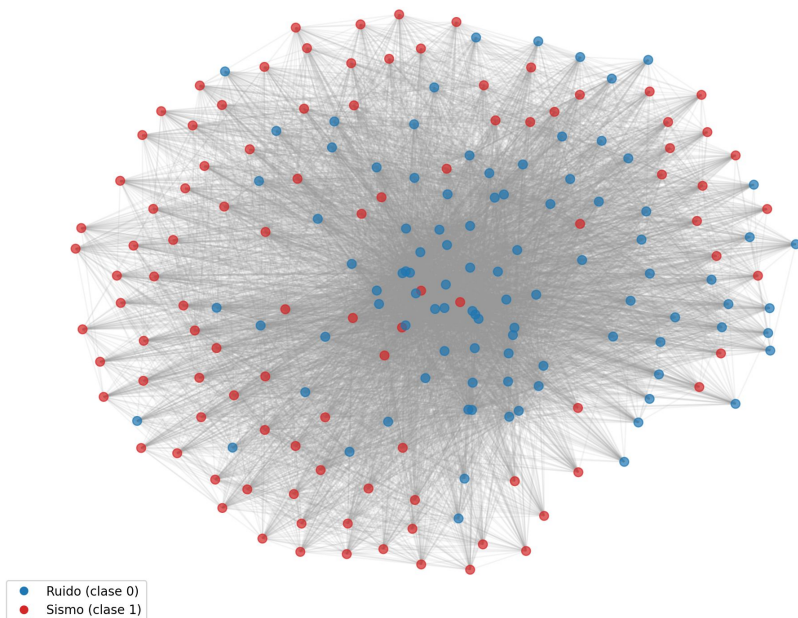
Evaluación

AUCROC es un indicador cuán bien el modelo discrimina y separa las distribuciones de ambas clases

Distribución de $P(\text{Sismo})$ en Test



Grafo de distancias bottleneck (layout de resortes)



Repositorio



Dado la complejidad del modelo y las pruebas, se optó por modularizar en clases de python y scripts dedicados. El repositorio es el siguiente:

<https://github.com/aaronbernal28/seismic-signals-tda>

Script de la búsqueda de hiperparametros:

https://github.com/aaronbernal28/seismic-signals-tda/blob/main/run_grid_search.py

Definición del modelo:

<https://github.com/aaronbernal28/seismic-signals-tda/blob/main/src/models/model2.py>

Reproducir resultados:

https://github.com/aaronbernal28/seismic-signals-tda/blob/main/scripts/06_best_model.py

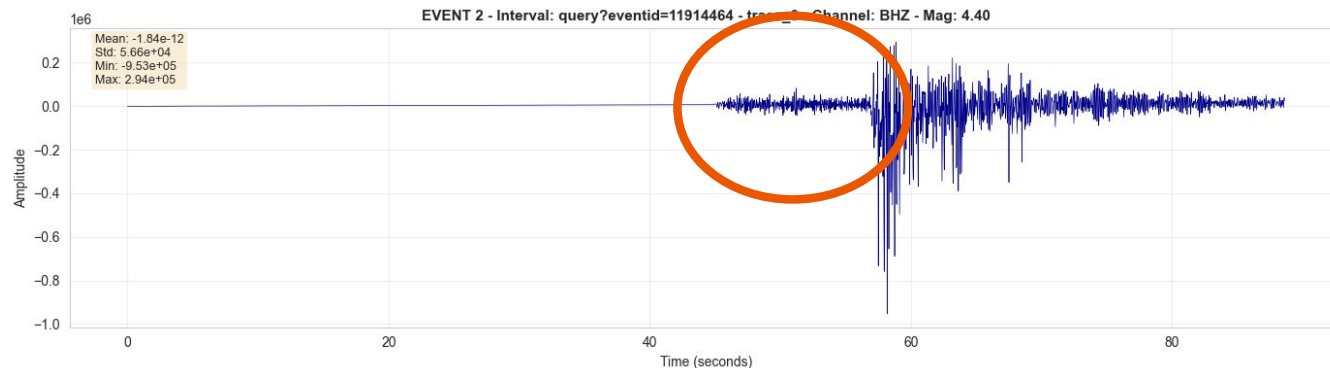
Proximo pasos...



- Paralelizar el cálculo el cálculo de distancias
- Implementar el algoritmo en C++
- Samplear de forma más inteligente, por ejemplo, los últimos más reciente o los más cercanos en distancia euclidea
- Incorporar las componentes ortogonales al eje vertical para obtener una solución más robusta
- H2

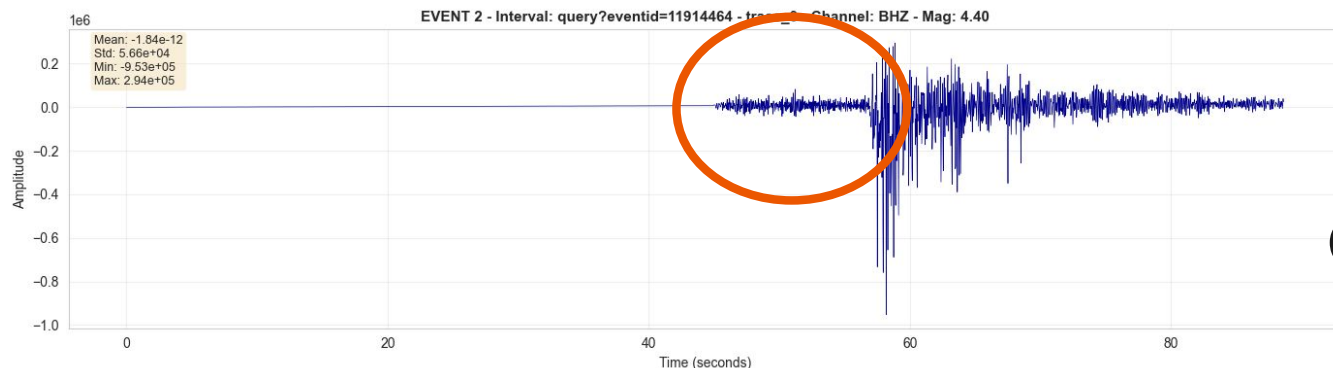
Proximo pasos...

- Paralelizar el cálculo el cálculo de distancias
- Implementar el algoritmo en C++
- Samplear de forma más inteligente, por ejemplo, los últimos más reciente o los más cercanos en distancia euclidea
- Incorporar las componentes ortogonales al eje vertical para obtener una solución más robusta
- Implementar la detección sobre los triggers, en vez de los eventos sismos, para apoyar los sistemas de alerta temprana



Proximo pasos...

- Paralelizar el cálculo el cálculo de distancias
- Implementar el algoritmo en C++
- Samplear de forma más inteligente, por ejemplo, los últimos más reciente o los más cercanos en distancia euclidea
- Incorporar las componentes ortogonales al eje vertical para obtener una solución más robusta
- Implementar la detección sobre los triggers, en vez de los eventos sismos, para apoyar los sistemas de alerta temprana



Gracias!